



Forecast Studio  
Antibiotic resistance

# How the Studio works

How data goes in, how the forecasts are made, and how the app is built: a guide to Antibiotic Resistance Forecast Studio.

Inputs & validation

12 forecasting models

ARIMA · SARIMA · LSTM · SARIMA-LSTM in depth

Architecture & packaging

Version 1.0.0 · October 2026

**Forecast Studio**  
Antibiotic resistance

WORKFLOW

✓ Add data  
Upload or type it in

✓ Review  
8 organisms · 2022–2025

3 Forecast  
4 models

DATA SET

antibiotic trend (2022–2025)  
8 organisms · 95 medicines ·  
2022–2025

● Confirmed

Studio > Forecast

STEP 3 OF 3

Forecast

What SARIMA-LSTM predicts from “antibiotic trend (2022–2025)” (2022–2025). Pick an organism, a medicine and a year.

 Comparing 4 models  
ARIMA · SARIMA · LSTM · SARIMA-LSTM

Model

Organism

Medicine

SARIMA-LSTM

Escherichia coli

Meropenem

Forecast year

2030

Recorded in 2025

31.9%

▲ +13.3 pts since 2022

SARIMA-LSTM forecast · 2030

46.1%

▲ +14.2 pts vs 2025

SARIMA-LSTM's average

17.0 pts

Worse than repeating the  
(pts)

Meropenem in Escherichia coli

Resistance recorded 2022–2025 and forecast to 2030 (5 years ahead)

Recorded

SARIMA-LSTM forecast

Show as table

How accurate SARIMA-LSTM

It learned from 2022–2024,  
for every medicine. The bar  
in percentage points: shorte

# Contents

## INTRODUCTION

|   |                                   |   |
|---|-----------------------------------|---|
| 1 | The Studio at a glance .....      | 3 |
| 2 | How the pieces fit together ..... | 4 |

## PART A Giving the models their inputs

|   |                                                           |    |
|---|-----------------------------------------------------------|----|
| 3 | Step 1 — Adding data .....                                | 6  |
| 4 | Step 2 — Reviewing and fixing the data .....              | 8  |
| 5 | Confirming: from the grid to a model-ready workbook ..... | 10 |

## PART B How the app predicts

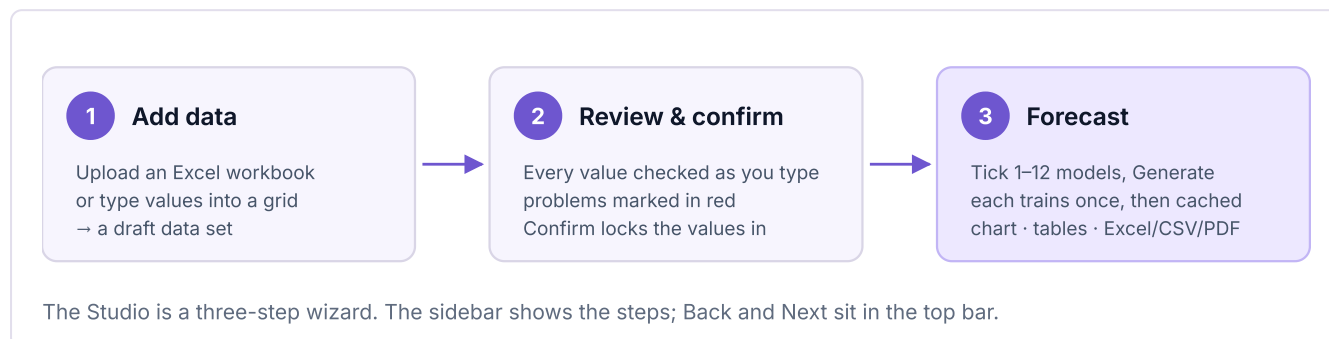
|    |                                                         |    |
|----|---------------------------------------------------------|----|
| 6  | Step 3 — Choosing models and generating forecasts ..... | 12 |
| 7  | How every model is trained and rolled forward .....     | 14 |
| 8  | How accuracy is measured .....                          | 16 |
| 9  | ARIMA in detail .....                                   | 18 |
| 10 | SARIMA in detail .....                                  | 19 |
| 11 | LSTM in detail .....                                    | 20 |
| 12 | The SARIMA-LSTM hybrid in detail .....                  | 22 |
| 13 | The four models on one medicine .....                   | 23 |
| 14 | The other eight models in brief .....                   | 24 |
| 15 | Reading and exporting the forecasts .....               | 25 |

## PART C How the app is made

|    |                                               |    |
|----|-----------------------------------------------|----|
| 16 | Technology stack .....                        | 27 |
| 17 | The desktop window and the local server ..... | 28 |
| 18 | The page — studio.html .....                  | 30 |
| 19 | Where the Studio keeps its data .....         | 31 |
| 20 | Building and packaging the installer .....    | 32 |
| 21 | Running it from source .....                  | 33 |
| 22 | Glossary .....                                | 33 |

## 1 The Studio at a glance

Antibiotic Resistance Forecast Studio is a Windows desktop app that turns a few years of resistance data — the percentage of each organism's isolates resistant to each antibiotic — into forecasts up to ten years ahead. The user brings the data, by uploading an Excel workbook or typing it in, checks it, and then chooses which of 12 forecasting models to run.



### INPUT

Yearly resistance percentages per organism and antibiotic, for at least 4 consecutive years

### MODELS

12 in four families: 5 statistical, 4 machine learning, 2 deep learning and 1 hybrid

### OUTPUT

A forecast for each of the 10 years after the last recorded year, kept within 0–100%

### ACCURACY

A backtest: each model predicts the last recorded year from the years before it

### RUNS

On one PC, offline: the window talks to a server on 127.0.0.1 that only this PC can reach

### BUILT WITH

Python 3.11, FastAPI, pywebview (Edge WebView2), statsmodels, scikit-learn, PyTorch and more; packaged with PyInstaller and Inno Setup

## About the examples

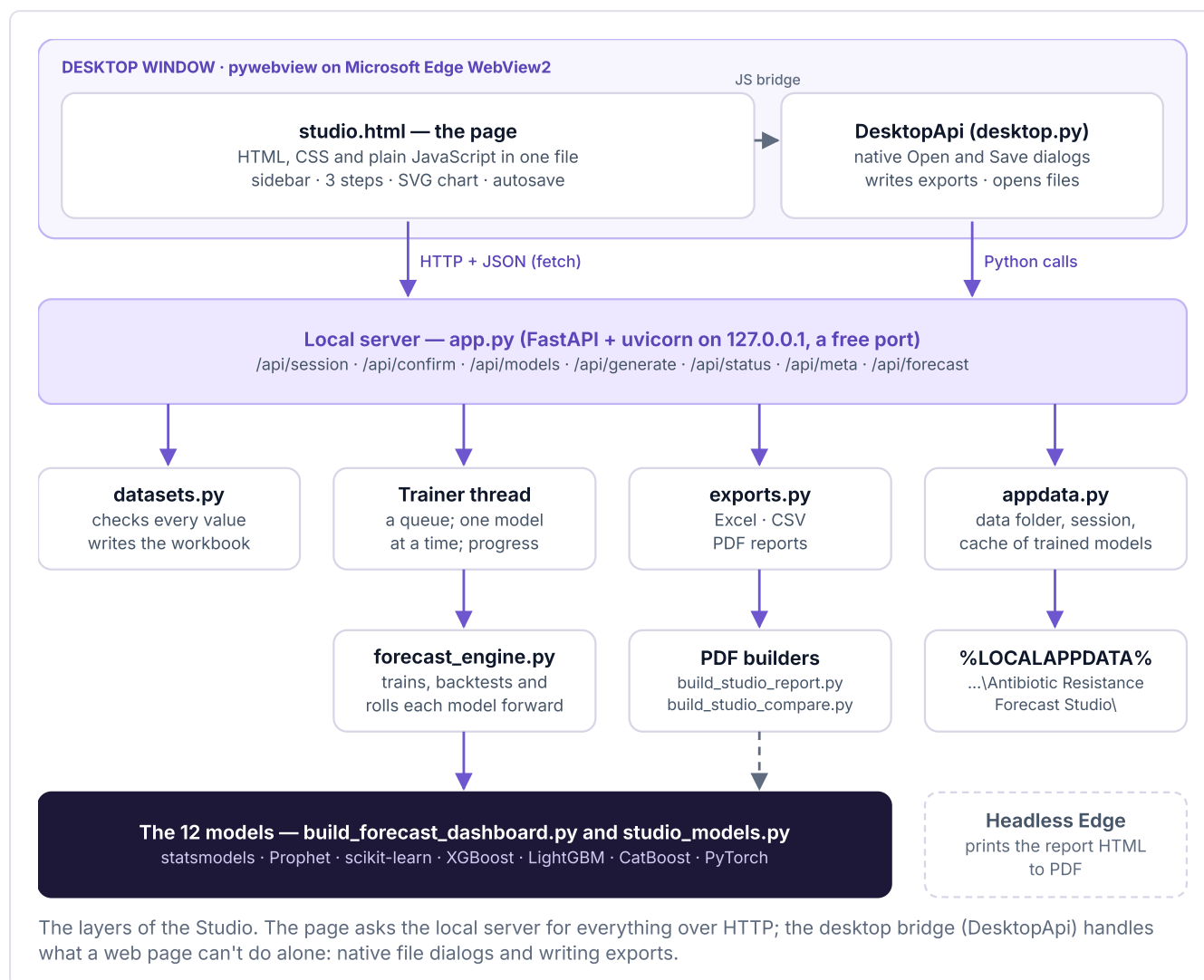
Every screenshot and number in this guide comes from the Studio itself, run on the sample workbook in the project folder, *antibiotic trend (2022-2025).xlsx*: 8 organisms (7 with data), 95 organism–antibiotic series, 2022–2025. One series is followed throughout: **Meropenem resistance in *E. coli***, recorded at 18.6% → 6.8% → 11.4% → 31.9%.

## How this guide is organised

- **Part A** follows the data in: adding it, reviewing it, and what Confirm does with it.
- **Part B** follows it through the models. ARIMA, SARIMA, LSTM and the SARIMA-LSTM hybrid are explained in full, with worked numbers. The other eight get a short entry each.
- **Part C** explains how the app is built: its parts, its storage, and its installer.

## 2 How the pieces fit together

The Studio is a small web application that runs entirely inside a desktop window. The page you see is HTML and JavaScript; behind it, a Python server on the same PC checks the data, trains the models and serves the forecasts.



### What happens when the app starts

1. **Antibiotic Resistance Forecast Studio.exe** runs `entry_desktop.py`, which selects the `entry` profile ( `appdata.use_profile("entry")` ): the app's name, its page ( `static/studio.html` ) and its own data folder.
2. `desktop.py` sends all output to `logs\\app.log`, checks that the Microsoft Edge WebView2 runtime is installed (offering Microsoft's download page if not), and stops child processes from flashing console windows.
3. It starts the FastAPI server ( `app.py` ) with uvicorn on a free port on 127.0.0.1, in a background thread, and waits until it is listening.
4. On start-up the server's `Trainer` reopens the last confirmed data set and any models already trained for it, from the cache. On first run there is nothing to open.
5. pywebview opens a 1280 × 860 window on the page (maximised if that doesn't fit the screen), with its background already in the theme's colour so it doesn't flash white in dark mode.
6. The page fetches `/api/status`, `/api/session` and `/api/models` together and reopens on the step the user left.

## The files that make up the Studio

| File                                                 | What it does                                                                         | Lines |
|------------------------------------------------------|--------------------------------------------------------------------------------------|-------|
| <code>entry_desktop.py</code>                        | Start-up: chooses the Studio profile, then runs the desktop shell                    | 21    |
| <code>desktop.py</code>                              | The window, native dialogs, exports, logging, and the build's self-test              | 461   |
| <code>app.py</code>                                  | The local server: every <code>/api/...</code> endpoint, and the <code>Trainer</code> | 392   |
| <code>datasets.py</code>                             | The data set format: read for review, check, clean up, write as a workbook           | 219   |
| <code>parse_antibiotic_trend.py</code>               | The strict workbook reader the models use; name clean-up tables                      | 232   |
| <code>forecast_engine.py</code>                      | Trains models, backtests them, rolls them forward, saves and reopens them            | 281   |
| <code>build_forecast_dashboard.py</code>             | Ten of the models and their settings (shared with the original Forecast app)         | 288   |
| <code>studio_models.py</code>                        | The two Studio-only models: SARIMA and the SARIMA-LSTM hybrid                        | 93    |
| <code>appdata.py</code>                              | Profiles, the data folder, the session, and the cache of trained models              | 150   |
| <code>exports.py</code>                              | Excel and CSV exports; calls the two PDF builders                                    | 144   |
| <code>build_studio_report.py</code>                  | The one-model PDF report                                                             | 449   |
| <code>build_studio_compare.py</code>                 | The comparison PDF report                                                            | 533   |
| <code>static/studio.html</code>                      | The whole user interface, with the Inter font embedded                               | 2,519 |
| <code>build_app.ps1</code> , <code>packaging\</code> | The build: PyInstaller spec, Inno Setup script, icon and font helpers                | —     |

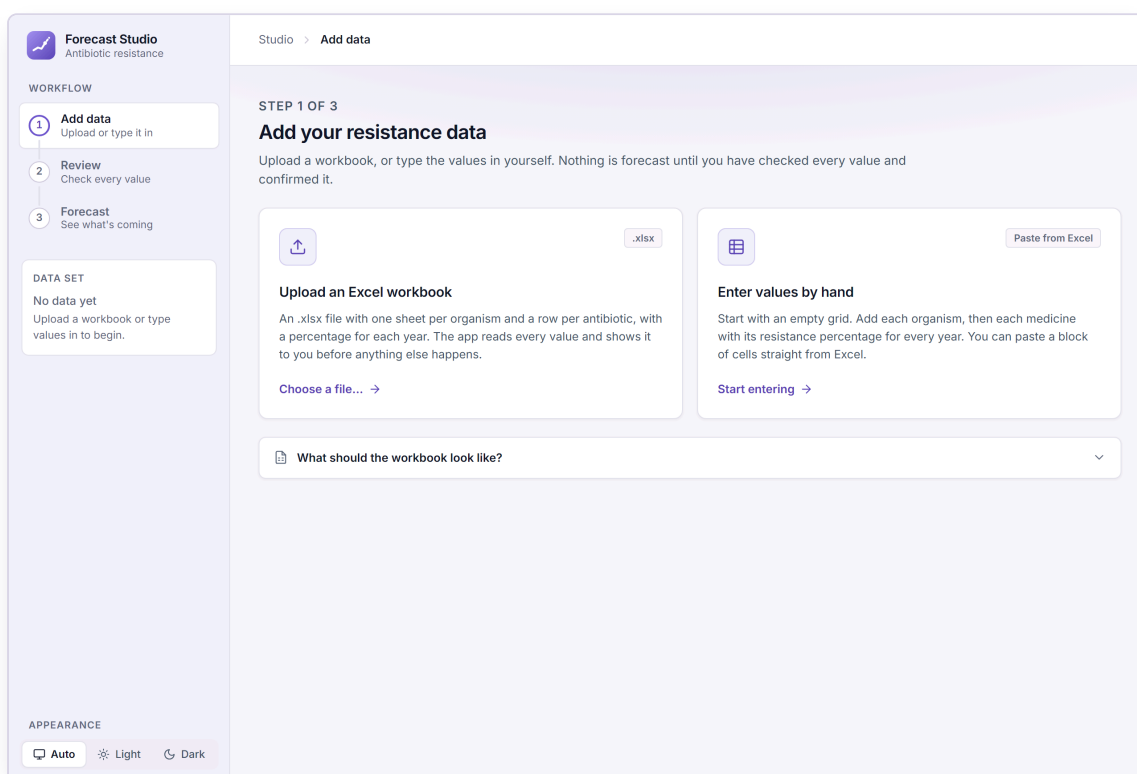
## PART A

# Giving the models their inputs

How resistance data gets into the Studio — uploaded or typed — how every value is checked, and what confirming turns it into before any model sees it.

### 3 Step 1 — Adding data

The Studio starts empty. Step 1 offers two ways in, and both end in the same place: a *draft data set* that Step 2 shows for review.



Step 1 on first launch. Once there is a draft, a “Continue with ...” card appears above the two choices.

### Route 1 — Upload a workbook

In the desktop app, **Upload a workbook** opens the Windows Open dialog ( `DesktopApi.read_workbook_for_review` ). In a browser the page uses a file input and posts the file to `/api/upload` . Either way the server runs `datasets.from_workbook()` . This reads every cell for review and does not reject the file over a bad value: it keeps the text so Step 2 can show it and you can fix it. Only a layout it can't make sense of is refused, with a message saying what to change.

## What the workbook must look like

- **One sheet per organism.** The sheet name is the organism: `E.COLI` becomes *Escherichia coli*; other all-capitals names are put in title case.
- **A header row starting “Antibiotic”** within the first 10 rows.
- **Year columns** right after it, written like `2022 (%)` : consecutive years, and the same years on every sheet. A `Net Change (%)` column after them is optional and not used by the models.
- **One row per antibiotic**, holding Excel percentages (a cell showing 25% stores 0.25) or text like `25%` .
- A row with a name but no numbers is read as a **note** on the organism. In the sample, *Streptococcus* has only “No antibiotics showed an increase...”, so it is listed as not forecast.

## One sheet of the sample workbook

### E. coli – Resistance Progression (2022 to 2025)

| Antibiotic                 | 2022 (%) | 2023 (%) | 2024 (%) | 2025 (%) |
|----------------------------|----------|----------|----------|----------|
| Cefaprazone with sulnactum | 20.93%   | 18.18%   | 29.55%   | 76.60%   |
| Ceftriaxone                | 0.00%    | 0.00%    | 13.64%   | 44.68%   |
| Meropenam                  | 18.60%   | 6.82%    | 11.36%   | 31.91%   |
| ...                        |          |          |          |          |

Misspelt drug names are mapped to standard ones (8 mappings, e.g. Meropenam → Meropenem, Amoxyclav → Amoxicillin-clavulanate, Gentamycin → Gentamicin); the review shows the original as “In the file: ...”.

## Route 2 — Type it in

**Type it in** starts a blank data set covering the last four full years (2022–2025 in 2026), with one organism and one empty row. **Add organism** and **Add medicine** grow it, and the year boxes add or remove columns. Cells accept `31.9` , `31.9%` or a decimal comma ( `31,9` ). A block of cells copied from Excel can be pasted in one go, and Enter moves down a row, as in a spreadsheet.

Forecast Studio

Antibiotic resistance

WORKFLOW

✓ Add data

Upload or type it in

2 Review

1 organism · 2022–2025

3 Forecast

See what's coming

DATA SET

Ward A, 2022–2025

1 organism · 2 medicines · 2022–2025

Not confirmed yet

APPEARANCE

Auto Light Dark

Studio > Review

All changes saved < Add data Confirm & choose a model →

STEP 2 OF 3

Review your data

Check every value, fix anything marked in red, then confirm. Your changes are saved as you type.

Data set name

Ward A, 2022–2025

Years

2022 to 2025

2 problems to fix →

Save as Excel...

ORGANISMS

Escherichia coli 1

+ Add organism

Organism

Escherichia coli

Remove organism

|   | Medicine   | 2022 (%) | 2023 (%) | 2024 (%) | 2025 (%) |   |
|---|------------|----------|----------|----------|----------|---|
| 1 | Meropenem  | 18.6     | 6.82     | 11.36    | 31.91    | × |
| 2 | Gentamycin | 22.5     | 19%      | 20.1     |          | × |

+ Add medicine

“20.1” is not a number.

Percentages from 0 to 100 · Enter moves down · paste a block of cells from Excel

Typing a data set in. “19%” is accepted; “20.1” (a letter O) is marked, and focusing it shows why. The empty 2025 cell is the second of the two problems.

## 4 Step 2 — Reviewing and fixing the data

Whichever way the data came in, Step 2 shows all of it, organism by organism, in an editable grid, and checks every value as you type.

The review after uploading the sample workbook: 8 organisms and 95 medicines. Badges count each organism's medicines, or its problems once there are any.

### The data set

Under the grid, a data set is plain JSON. It is the same whether the values were uploaded or typed in, so the page can edit it and the app can save it between sessions:

```
{
  "name": "antibiotic trend (2022-2025)",
  "source": "upload",
  "file": "antibiotic trend (2022-2025).xlsx",
  "years": [2022, 2023, 2024, 2025],
  "organisms": [
    {
      "name": "Escherichia coli",
      "note": null,
      "rows": [
        {
          "medicine": "Meropenem",
          "original": "Meropenam",
          "values": [18.6, 6.82, 11.36, 31.91],
          ...
        }
      ]
    }
  ]
}
```

Values are percentages from 0 to 100. A cell that can't be used keeps what it held (`null` when empty, the text otherwise) so the review can show it.

### Saved as you type

0.3 seconds after the last keystroke the page sends the whole session — step, data set, theme and forecast view — with `PUT /api/session`. The server writes `session.json` through a temporary file, so a crash never leaves half of it, and replies with `datasets.problems()` for the data set. The page marks each problem on its field and counts them in the summary chip ("2 problems to fix →" jumps to the next one). Closing the app loses nothing: it reopens on the same step with the same values.

**Forecast Studio**  
Antibiotic resistance

**WORKFLOW**  

**Add data**  
Upload or type it in

**Review**  
8 organisms · 2022-2025

**Forecast**  
See what's coming

**DATA SET**  
**antibiotic trend (2022-2025)**  
8 organisms · 95 medicines · 2022-2025  

Not confirmed yet

**APPEARANCE**  

Auto

Light

Dark

Studio > Review

All changes saved < Add data Confirm & choose a model →

**STEP 2 OF 3**  
**Review your data**  
Check every value, fix anything marked in red, then confirm. Your changes are saved as you type.

Data set name

Years

antibiotic trend (2022-2025) 2022 to 2025

2 problems to fix →

Save as Excel...

**ORGANISMS**  

**Escherichia coli** 2

Klebsiella pneumoniae 14

Coagulase-negative Sta... 7

Acinetobacter spp. 17

Staphylococcus aureus 19

Streptococcus spp. no data

Enterococcus spp. 8

Pseudomonas aerugino... 15

+ Add organism

Organism

Escherichia coli Remove organism

|   | Medicine                                                          | 2022 (%) | 2023 (%) | 2024 (%) | 2025 (%) |   |
|---|-------------------------------------------------------------------|----------|----------|----------|----------|---|
| 1 | Cefoperazone-sulbactam<br>In the file: Cefaprazone with sulnactum | 20.93    | 18.18    | 29.55    | 76.6     | × |
| 2 | Ceftriaxone                                                       | 0        | 0        | 13.64    | 44.68    | × |
| 3 | Ertapenem                                                         | 16.28    | n/a      | 20.45    | 59.57    | × |
| 4 | Cefuroxime                                                        | 34.88    | 31.82    | 38.64    | 76.6     | × |
| 5 | Cefepime                                                          | 37.21    | 22.73    | 52.27    |          | × |
| 6 | Ciprofloxacin                                                     | 27.91    | 38.64    | 47.73    | 63.83    | × |
| 7 | Amoxicillin-clavulanate<br>In the file: Amoxycylav                | 0        | 0        | 9.09     | 34.04    | × |
| 8 | Imipenem                                                          | 18.6     | 6.82     | 20.45    | 51.06    | × |
| 9 | Gentamicin                                                        | 0        | 0        | 9.09     | 23.4     | × |

Two problems introduced on purpose: text in a value cell and an empty cell. Both are red, the organism's badge counts them, and Confirm is refused until they are fixed.

## Every check

| What     | Rule                                                              | Message shown                                                                                                             |
|----------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Years    | Written in full (1000–9999)                                       | "Years must be written in full, like 2022."                                                                               |
| Years    | At least 4                                                        | "Add at least 4 years: the models need two years to learn from, one to predict and one to check that prediction against." |
| Years    | Consecutive                                                       | "The years must follow on from each other."                                                                               |
| Data set | At least one medicine with values                                 | "Add at least one medicine with its percentages."                                                                         |
| Organism | Named; no [ ] : * ? / \ ; at most 31 characters; not listed twice | Excel's own rules for sheet names, since each organism becomes a sheet                                                    |
| Medicine | Named; not listed twice for one organism, after name clean-up     | "Meropenem is listed twice for this organism."                                                                            |
| Value    | Present, a number, within 0–100                                   | "Enter a percentage." · "'n/a' is not a number." · "120% is outside 0–100%."                                              |

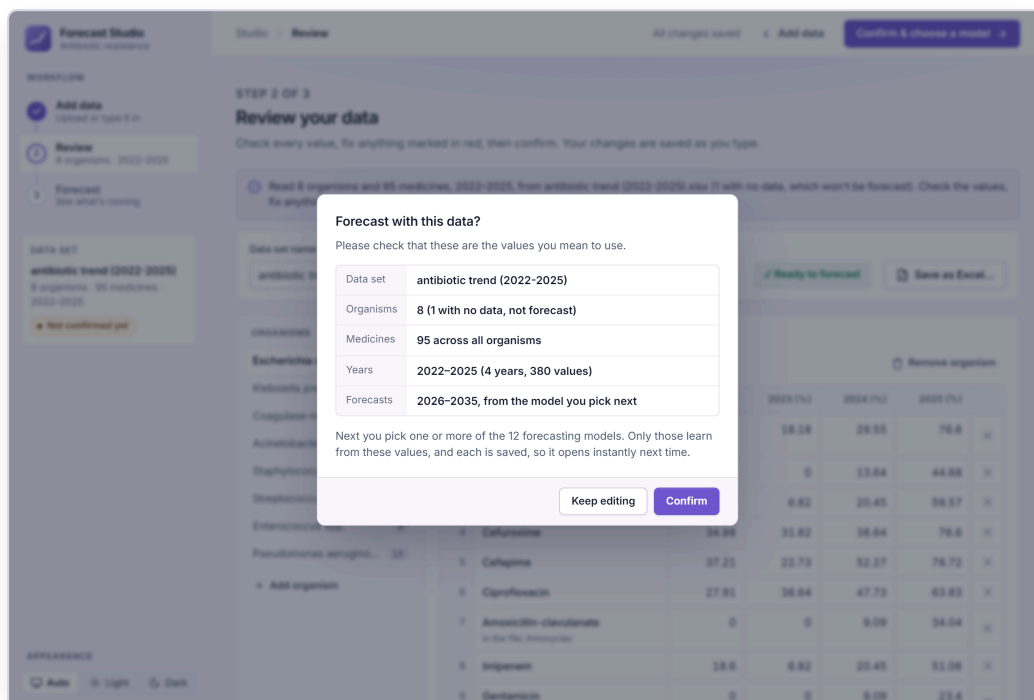
**Why four years?** Four years is the minimum because the learned models read the two years before the one they predict, and the backtest needs one more year to predict and one to check that prediction against.

Antibiotic Resistance Forecast Studio · How it works

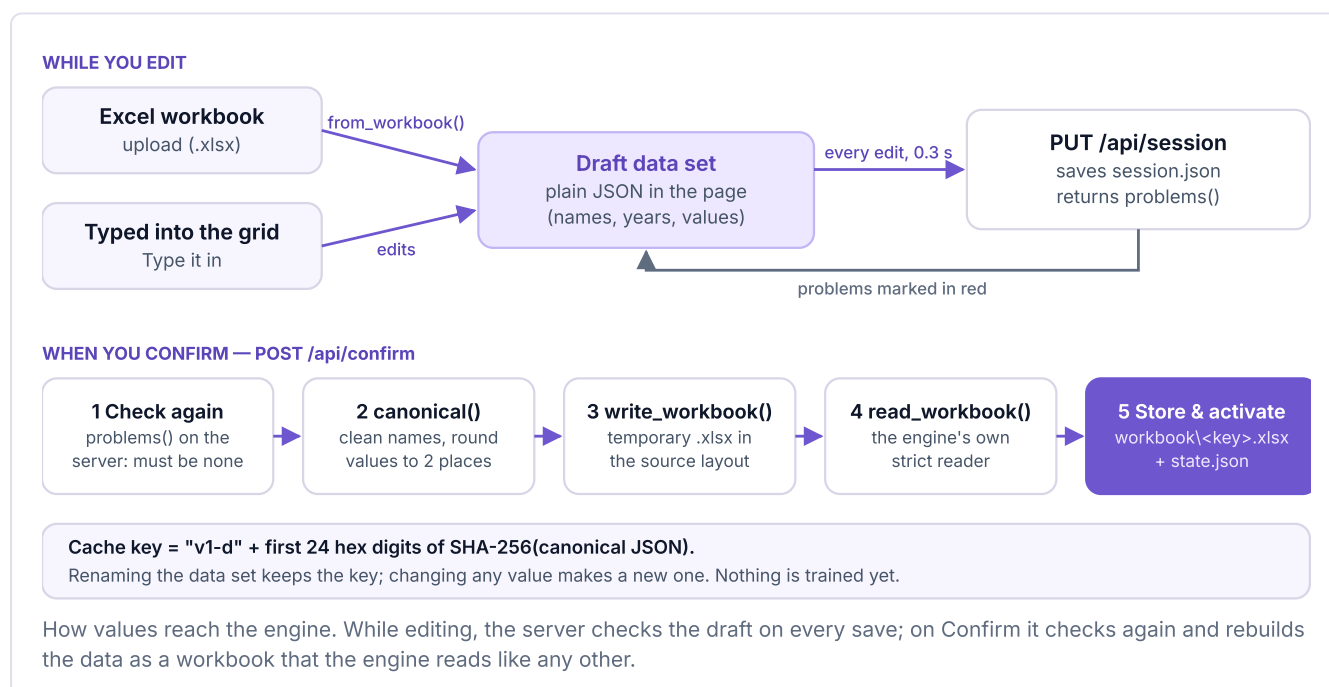
9 / 33

## 5 Confirming: from the grid to a model-ready workbook

Pressing **Confirm & choose a model** asks the user to check a summary, then turns the draft into the exact form the forecasting engine reads. Confirming does not train anything.



The confirmation dialog summarises what will be forecast: organisms, medicines, years and values, and the forecast years (2026–2035).



### What `POST /api/confirm` does

1. **Checks again.** The server runs `problems()` itself rather than trusting the page, and refuses the data if anything is left to fix.
2. **Cleans it up** with `canonical()`: drug and organism names are standardised, every value is turned into a number rounded to 2 decimal places, and an organism without rows gets a note.

3. **Writes a workbook** with `write_workbook()` : a temporary `.xlsx` in the source layout, with a sheet per organism, the header on row 4 and values stored as Excel percentages.
4. **Reads it back** with `read_workbook()` , the same strict reader the engine uses for training, so the models only ever see data that passed it.
5. **Names it** with a cache key (below), copies it to `workbook\<key>.xlsx` in the data folder, and records it in `workbook\state.json` as the active data set.
6. **Opens any models already trained** for exactly these values from `cache\<key>\` . In Step 3 they show as *Ready* and open instantly.

datasets.py

```
def dataset_key(dataset: dict) -> str:
    """Cache name for a data set: changes whenever its values or the models change, not with its name."""
    text = json.dumps(canonical(dataset), sort_keys=True, ensure_ascii=False)
    return f"v{CACHE_VERSION}-d{hashlib.sha256(text.encode()).hexdigest()[:24]}"
```

The key is a fingerprint of the cleaned values. Renaming a data set keeps the same key, so nothing retrains, while changing a single value gives a new key and fresh models. `CACHE_VERSION` (now 1) is raised whenever a model's settings change, so caches made with the old settings are not reused.

**Round trip.** Writing the data out as a workbook and reading it back means a workbook uploaded and confirmed unchanged gives exactly the same forecasts as training on the workbook itself. The build's self-test checks this every time: the largest difference must be 0.00 percentage points.

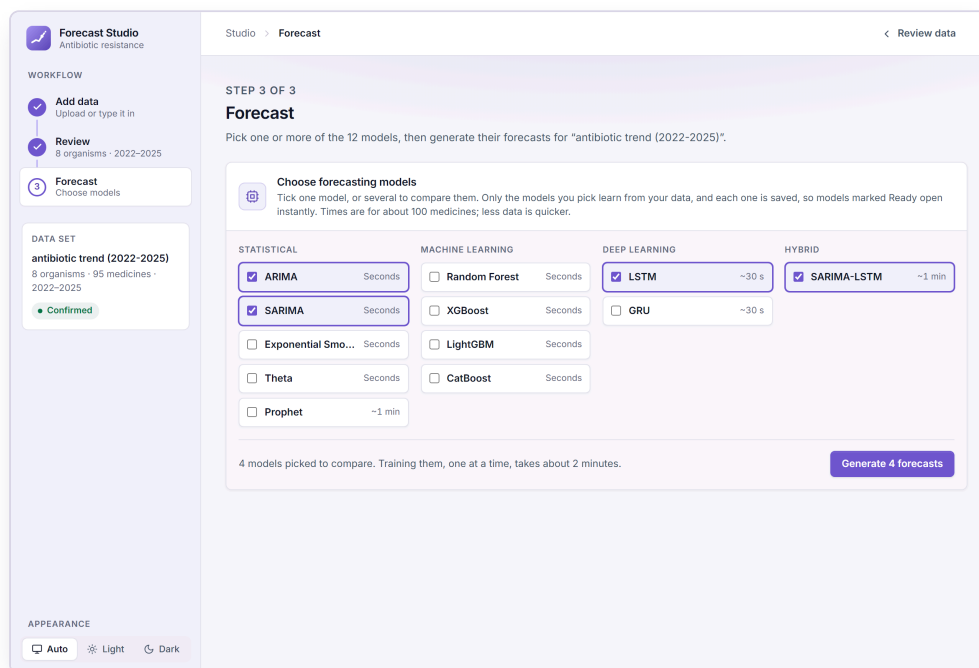
## PART B

# How the app predicts

What happens when you press Generate: how the 12 models learn from the data, how their accuracy is measured, and four of them — ARIMA, SARIMA, LSTM and the SARIMA-LSTM hybrid — step by step.

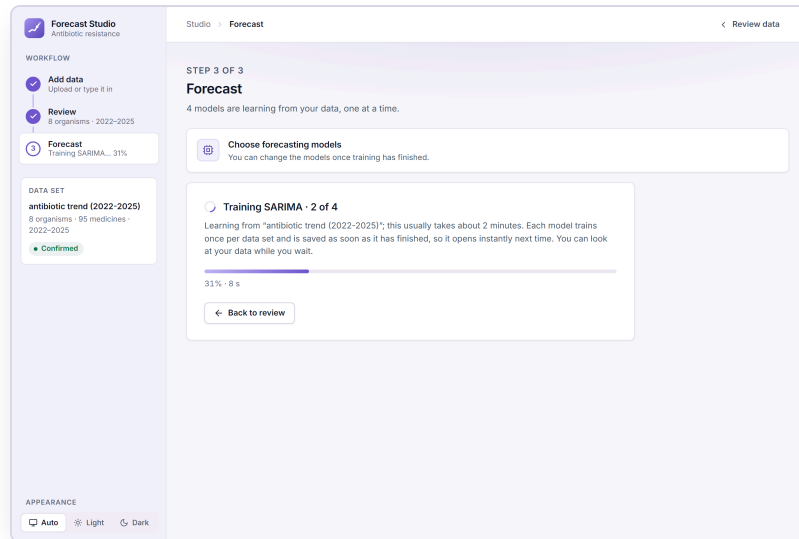
## 6 Step 3 — Choosing models and generating forecasts

Step 3 opens with the model picker. Only the models ticked here learn from the data, and each one is saved as soon as it has trained, so it never trains twice for the same values.

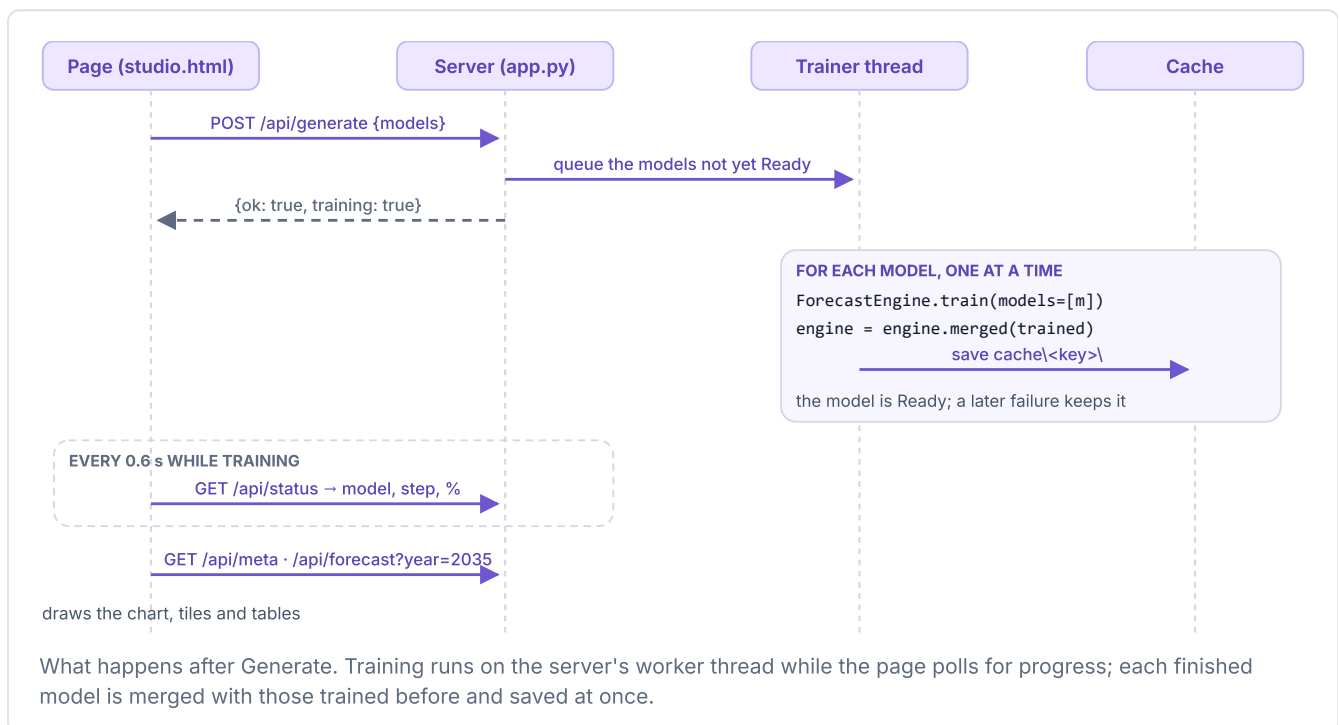


The model picker with the four models this guide covers in detail ticked. Each shows a rough training time for about 100 medicines; models already trained for this data show “Ready”.

The picker's list comes from `/api/models`: each model's name, family, a one-line description, and how long it took to train on the sample's 95 series. Ticking one model shows its forecast; ticking several adds a **Model** menu to switch between them and a comparison PDF to the exports. **Generate** trains only the ticked models that are not Ready yet.



Training. The card and the sidebar both show which model is learning, how far it has got, and the time so far; the data can still be browsed in Step 2 meanwhile.



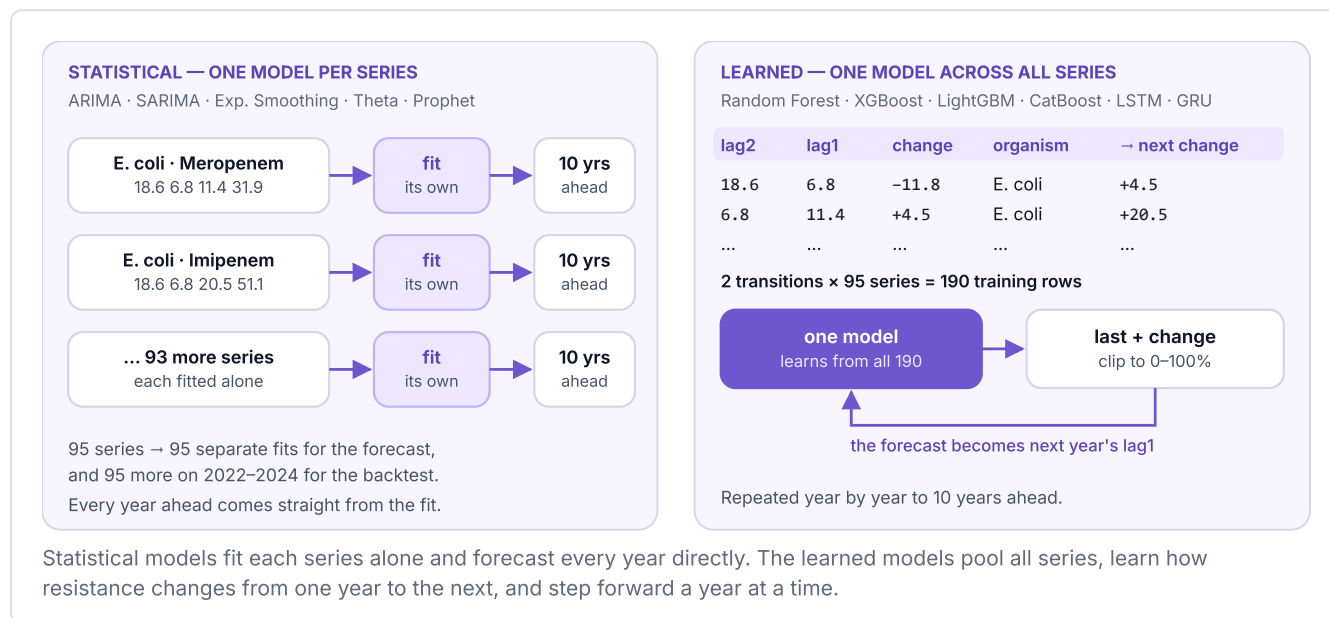
## The Trainer

- **One worker thread and a queue.** `/api/generate` only queues the job and returns at once. The worker trains the models one at a time, in the picker's order.
- **Saved as it goes.** Each model is trained on its own ( `ForecastEngine.train(models=[m])` ), merged into the models already trained on the same data ( `merged()` ), saved to the cache, and is Ready while the next one trains. If a later model fails, the earlier ones are kept; the card shows the error and **Try again**.
- **Progress.** Models report how far they have got ( `report(model, index, total, fraction)` ), and the page reads it from `/api/status` every 0.6 s.
- **Repeatable.** All training runs on one thread: PyTorch sets up its CPU threading per thread, and on a fresh thread LSTM forecasts moved by up to a point.
- **Independent.** A model forecasts the same trained alone or with others, which the build's self-test checks.

## 7 How every model is trained and rolled forward

The engine ( `forecast_engine.py` ) reads the confirmed workbook into a table of 95 series × 4 years and trains the requested models in one of two ways, depending on the model.

Each row of the table is one organism–antibiotic **series**, such as Meropenem in *E. coli*: [18.6, 6.82, 11.36, 31.91]. The organism comes along as a label; the 7 organisms with data become the *levels* the learned models can tell apart.



### Statistical models: one fit per series

ARIMA, SARIMA, Exponential Smoothing, Theta and Prophet look at one series at a time and extrapolate it 10 years. They are fitted twice per series: once on 2022–2024 for the backtest, once on all four years for the forecast.

### Learned models: one model across all series

Four values per series are far too few to train a machine-learning model on one series. So Random Forest, XGBoost, LightGBM, CatBoost, LSTM and GRU each learn one shared rule from all 95 series: *given the last two years and the organism, how much will resistance change next year?* Each year-to-year step of each series is one training row. Four years give two steps per series, so **190 rows**.

`build_forecast_dashboard.py` — the inputs every learned model sees

```
def lag_features(lag1, lag2, organisms, organism_levels) -> pd.DataFrame:
    lag1, lag2 = np.asarray(lag1, dtype=float), np.asarray(lag2, dtype=float)
    X = pd.DataFrame({"lag1": lag1, "lag2": lag2, "change": lag1 - lag2})
    for org in organism_levels:
        X[f"org_{org}"] = (np.asarray(organisms) == org).astype(float)
    return X
```

To forecast, the predicted change is added to the latest value, and that forecast becomes the newest year for the next step. This repeats for 10 years:

```
forecast_engine.py
```

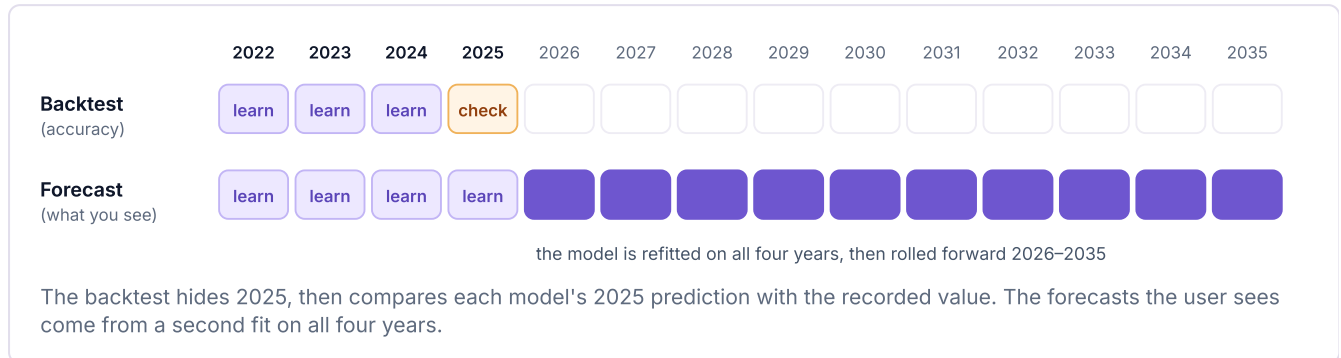
```
def roll_forward(model, prev, last, orgs, levels, steps):  
    """Add the predicted change to the latest value, one year at a time, keeping each year within 0-100%."""  
    path = []  
    for _ in range(steps):  
        nxt = np.clip(last + model.predict(lag_features(last, prev, orgs, levels)), 0, 100)  
        path.append(nxt)  
        prev, last = last, nxt  
    return np.column_stack(path)
```

## What the engine keeps

- **Ten years ahead** ( `YEARS_AHEAD = 10` ): 2026–2035 for the sample, every value kept within 0–100%.
- **Forecast paths** in one array, `paths[series, model, year]` ( $95 \times 12 \times 10$  once all 12 are trained), plus the recorded values and a `meta.json` with each model's accuracy. Every year in range is a slice of this array, so moving the year slider never retrains.
- **A safe fallback.** Typed-in data can be too small or too flat for some libraries: LightGBM needs more than one training row, and CatBoost refuses a target that never changes. `fit_learned()` then substitutes an average-change model and says so in that model's description, rather than failing.

## 8 How accuracy is measured

Every model is tested the same way before its forecasts are shown: it is trained without the last recorded year and asked to predict it.



Two scores come out of the backtest, both in percentage points (pts) across all 95 series:

**MAE**, the average miss

$$\text{MAE} = (1/n) \sum |\text{predicted} - \text{recorded}|$$

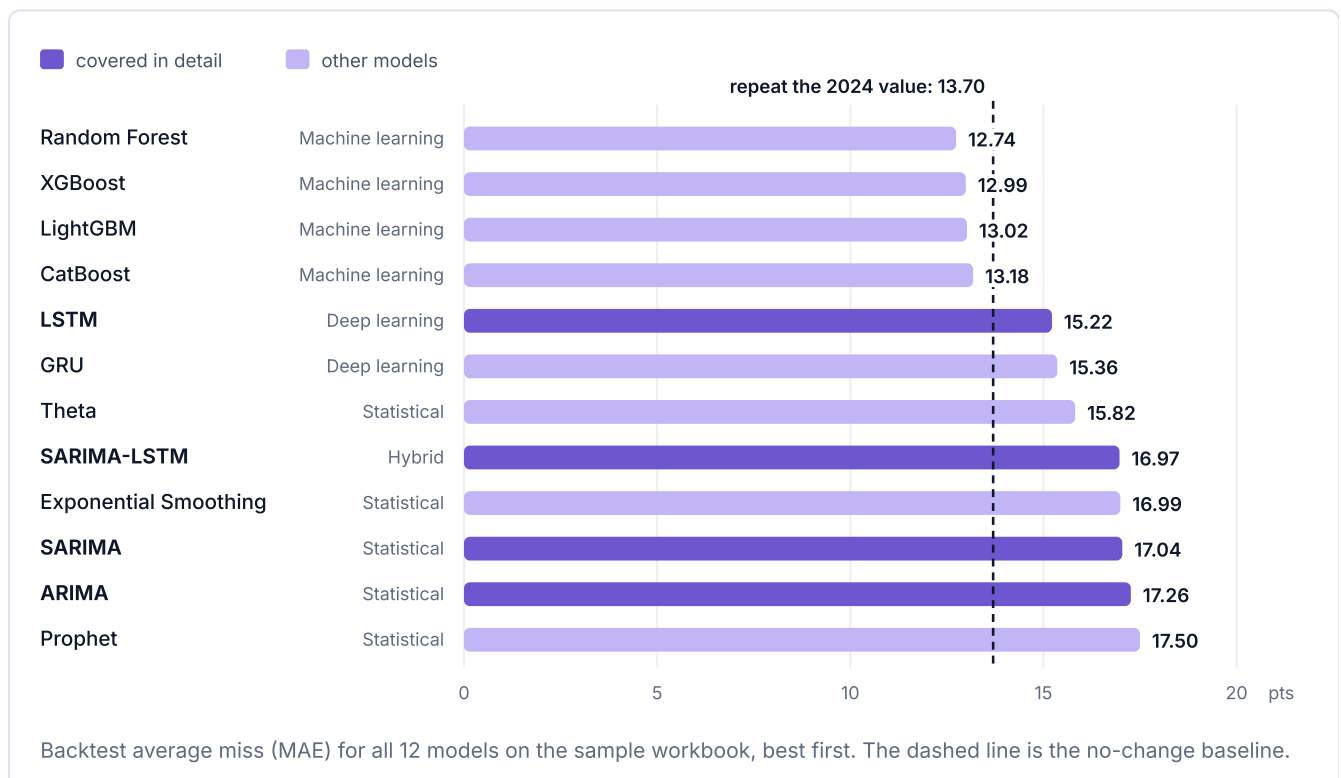
What the app shows as "average miss". 15 pts means a typical 2025 prediction was 15 points out.

**RMSE**, which weighs large misses more

$$\text{RMSE} = \sqrt{(1/n) \sum (\text{predicted} - \text{recorded})^2}$$

Shown in the Excel export and the PDF reports.

Both are compared with a **baseline** that needs no model at all: *repeat the 2024 value*. A model is only useful if it beats that. The Studio's most accurate model is the one with the lowest MAE.



**Read these numbers with care.** On this data only the four tree-based models beat simply repeating the previous year (13.70 pts); Random Forest is best at 12.74. The four models this guide covers in detail miss by 15.22–17.26 pts. That reflects how little history there is (one test year, four values per series) rather than a fault: the app's notes say forecasts beyond a year or two should be treated as rough.

## All twelve models

| Model                        | Family               | MAE          | RMSE         | Rank      |
|------------------------------|----------------------|--------------|--------------|-----------|
| Random Forest                | Machine learning     | 12.74        | 16.16        | 1         |
| XGBoost                      | Machine learning     | 12.99        | 16.51        | 2         |
| LightGBM                     | Machine learning     | 13.02        | 16.55        | 3         |
| CatBoost                     | Machine learning     | 13.18        | 16.58        | 4         |
| <b>LSTM</b>                  | <b>Deep learning</b> | <b>15.22</b> | <b>19.27</b> | <b>5</b>  |
| GRU                          | Deep learning        | 15.36        | 19.37        | 6         |
| Theta                        | Statistical          | 15.82        | 19.55        | 7         |
| <b>SARIMA-LSTM</b>           | <b>Hybrid</b>        | <b>16.97</b> | <b>21.01</b> | <b>8</b>  |
| Exponential Smoothing        | Statistical          | 16.99        | 20.99        | 9         |
| <b>SARIMA</b>                | <b>Statistical</b>   | <b>17.04</b> | <b>20.79</b> | <b>10</b> |
| <b>ARIMA</b>                 | <b>Statistical</b>   | <b>17.26</b> | <b>21.15</b> | <b>11</b> |
| Prophet                      | Statistical          | 17.50        | 21.24        | 12        |
| <i>Repeat the 2024 value</i> | <i>Baseline</i>      | <i>13.70</i> | —            | —         |

ARIMA treats next year as *this year, plus a steady yearly drift, plus noise*. In the Studio it is the simplest trend model there is: a random walk with drift.

### The model

ARIMA( $p, d, q$ ) has  $p$  autoregressive terms,  $d$  rounds of differencing and  $q$  moving-average terms. The Studio uses **ARIMA(0, 1, 0) with drift**: differencing once turns the series into its yearly changes, and with no AR or MA terms, those changes are modelled as a constant  $c$  plus random noise:

$$y_t = y_{t-1} + c + \varepsilon_t \Rightarrow \hat{y}_{T+h} = y_T + h \cdot c$$

Fitted by maximum likelihood, the drift  $c$  is the average yearly change,  $(\text{last} - \text{first}) \div (\text{years} - 1)$ . The forecast is a straight line from the last recorded value. This is the largest ARIMA that 3–4 points can support: every extra AR or MA term is another parameter to estimate from the same handful of numbers.

```
build_forecast_dashboard.py (statsmodels)
```

```
def arima(history, steps):
    # (0,1,0) with drift is the largest ARIMA order 3-4 points can support.
    return ARIMA(history, order=(0, 1, 0), trend="t").fit().forecast(steps)
```

With one difference, `trend="t"` (a linear trend in the levels) is exactly the constant drift in the yearly changes.

### Worked example: Meropenem in *E. coli*

|               | 2022 | 2023   | 2024  | 2025   |
|---------------|------|--------|-------|--------|
| Recorded      | 18.6 | 6.82   | 11.36 | 31.91  |
| Yearly change |      | -11.78 | +4.54 | +20.55 |

Drift  $c = (31.91 - 18.6) \div 3 = \mathbf{+4.44 \text{ pts a year}}$ , so the forecast climbs by that much every year:

| Year           | 2026  | 2027  | 2028  | 2030  | 2035  |
|----------------|-------|-------|-------|-------|-------|
| ARIMA forecast | 36.3% | 40.8% | 45.2% | 54.1% | 76.3% |

**The backtest for this series** only had 2022–2024: changes of -11.78 and +4.54, so a drift of -3.62 a year, and a predicted 2025 of **7.7%**. The recorded value was 31.9%: a miss of 24.2 points.

**Strengths.** Transparent and fast; sensible for a series that really is moving steadily one way; never needs other series.

**Limits here.** The line is set by the first and last points alone, so one unusual year swings it. It never levels off, reaching 76.3% by 2035 here, and it learns nothing from the other 94 series.

## 10 SARIMA in detail

Statistical

Per series · Studio only

Backtest MAE 17.04 · RMSE 20.79 · rank 10 of 12

≈ 11 s for 95 series

SARIMA is ARIMA with seasonal terms. In the Studio its real difference from ARIMA is that it **chooses its own form for each series** rather than always assuming a drift.

### The model and how its form is chosen

The Studio fits SARIMA with statsmodels' `SARIMAX`. Seasonal terms describe a cycle within the year, such as winter peaks, and yearly data has no such cycle, so they are switched off, as automatic SARIMA tools do for yearly data. What remains is a search: several candidate orders are fitted, each with and without drift, and the one with the lowest **AIC** wins.  $AIC = 2k - 2 \ln L$ , where  $k$  is the number of parameters and  $L$  is how likely the data are under the fitted model; lower is better, so an extra parameter must improve the fit enough to pay for itself.

Every series is tried as a random walk, (0,1,0). From 8 values on, (1,1,0), (0,1,1) and (1,1,1) are tried as well. Each order is fitted with and without drift: 2 fits per series, or 8.

studio\_models.py (warnings handling trimmed)

```
def sarima_fit(history):
    orders = [(0, 1, 0)]
    if len(history) >= ARMA_MIN_VALUES:          # 8
        orders += [(1, 1, 0), (0, 1, 1), (1, 1, 1)]
    fits = [SARIMAX(np.asarray(history, dtype=float), order=order, trend=trend).fit(dispatch=False)
            for order in orders for trend in ("n", "c")] # with d=1, "c" is the drift
    return min(fits, key=lambda fit: fit.aic)
```

Why wait for 8 values? With fewer, AR and MA terms fit the noise; on the sample they forecast 105% for 2026 for a series recorded at 77%. So on four years SARIMA chooses between a **flat forecast** (the last value carried forward) and **ARIMA's straight line**.

### Worked example: Meropenem in *E. coli*

| Candidate          | AIC   | Forecast                                      |
|--------------------|-------|-----------------------------------------------|
| (0,1,0), no drift  | 26.32 | 31.91% every year — chosen                    |
| (0,1,0) with drift | 27.99 | ARIMA's line: 36.3% in 2026 ... 76.3% in 2035 |

The yearly changes (-11.8, +4.5, +20.6) are so uneven that a drift doesn't improve the fit enough to pay for its extra parameter. SARIMA therefore forecasts Meropenem resistance in *E. coli* to stay at 31.9% for all ten years, where ARIMA had it climbing to 76.3%.

### Across the sample

- For the forecasts (four years), AIC chose **no drift for 76** of 95 series and drift for 19.
- In the backtest (three years) it chose drift for 26 series. On those, SARIMA missed 2025 by 22.7 pts on average, where repeating 2024 missed by 10.4; on the other 69 it *is* the repeat-2024 baseline. Overall MAE 17.04, a little better than ARIMA's 17.26.

**Strengths.** Takes the simpler model unless the data justify a trend; a flat forecast is often a sensible default for noisy series; ready to use AR and MA terms once there are 8 or more years.

**Limits here.** With 4 yearly values it can only be a random walk, with or without drift, so it forecasts much like ARIMA or like "no change". It has no seasonal part to use on yearly data.

## 11 LSTM in detail

Deep learning

One network across all series

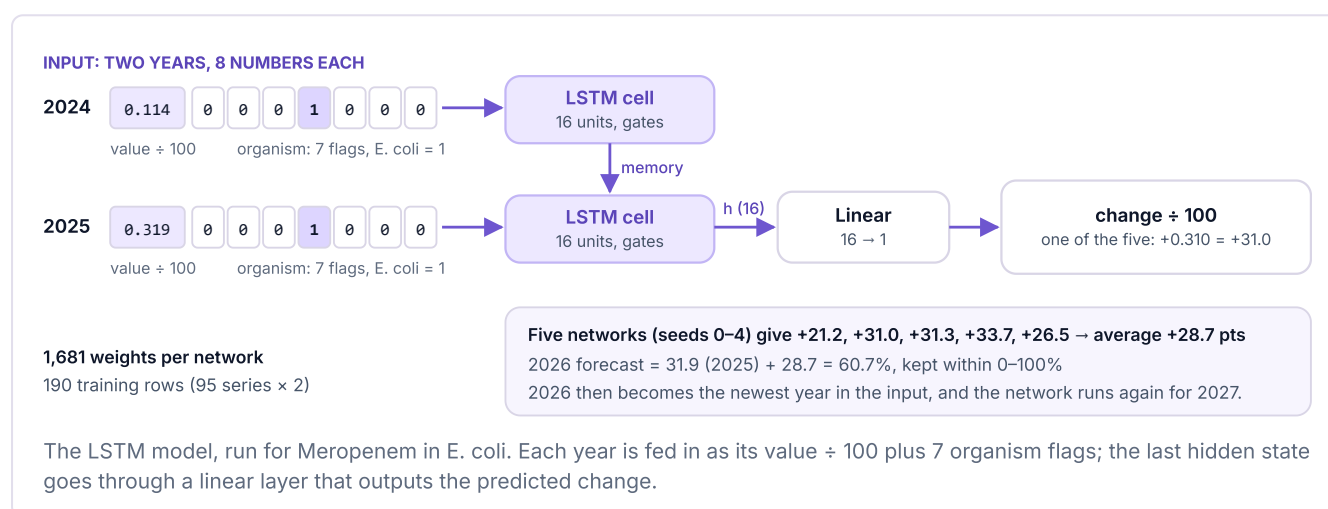
Backtest MAE 15.22 · RMSE 19.27 · rank 5 of 12

≈ 38 s for 95 series

An LSTM (long short-term memory) network is a neural network that reads a sequence one step at a time and keeps a memory between steps. In the Studio it reads the last two years of a series, with its organism, and predicts next year's change.

### How an LSTM works, briefly

At each step an LSTM cell takes the new input and its memory so far (the *cell state*). Three learned **gates**, each a small layer of numbers between 0 and 1, decide what to forget from memory, what to add from the new input, and what to output. This lets it pick up patterns in sequences, such as “after a sharp rise, resistance tends to keep rising”. Here the sequence is short, two years, but the same network is trained on every series at once, so it learns from all 95.



### How the Studio sets it up

| Setting       | Value                                                                                           | Why                                                                               |
|---------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Input         | 2 time steps × 8 numbers: value ÷ 100, and 7 organism flags                                     | Two years is all the lag 4 years allow; scaling keeps the network's numbers small |
| Network       | LSTM with 16 hidden units → linear layer 16 → 1; 1,681 weights                                  | Small, to suit a small data set                                                   |
| Target        | Next year's change ÷ 100                                                                        | Changes are similar across series; levels are not                                 |
| Training rows | 190 (95 series × 2 steps); 95 for the backtest                                                  | Every step of every series                                                        |
| Training      | 400 passes over all rows; Adam, learning rate 0.01, weight decay $10^{-4}$ ; mean squared error | Weight decay keeps weights small against overfitting                              |
| Ensemble      | 5 networks (seeds 0–4), predictions averaged                                                    | On ~200 rows a network depends heavily on its starting weights                    |
| Forecast      | Last value + predicted change, kept within 0–100%, fed back for the next year                   | The shared roll-forward of the learned models                                     |

build\_forecast\_dashboard.py (PyTorch, CPU)

```
class _RecurrentNet(torch.nn.Module):
    def __init__(self, cell, n_inputs, hidden):
        super().__init__()
        self.rnn = {"lstm": torch.nn.LSTM, "gru": torch.nn.GRU}[cell](n_inputs, hidden, batch_first=True)
        self.head = torch.nn.Linear(hidden, 1)

    def forward(self, seq):
        out, _ = self.rnn(seq)
        return self.head(out[:, -1])          # the hidden state after the last year

# RecurrentRegressor.fit, for each of the five seeds:
torch.manual_seed(seed)
net = _RecurrentNet(self.cell, seq.shape[2], self.hidden)
opt = torch.optim.Adam(net.parameters(), lr=self.lr, weight_decay=1e-4)
for _ in range(self.epochs):                # 400
    opt.zero_grad()
    torch.nn.functional.mse_loss(net(seq), target).backward()
    opt.step()
```

### Worked example: Meropenem in *E. coli*

For 2026 the network reads 2024 (0.1136) and 2025 (0.3191), both flagged *E. coli*. The five networks predict changes of +21.2, +31.0, +31.3, +33.7, +26.5 points. The spread, about 12 points, is why the Studio averages them: their mean, **+28.7**, gives 2026 = 31.9 + 28.7 = **60.7%**. The 2026 value is then fed back as the newest year:

| Year | 2026 | 2027 | 2028 | 2029 | 2030 | 2031 | 2032 | 2033 | 2034 | 2035 |
|------|------|------|------|------|------|------|------|------|------|------|
| LSTM | 60.7 | 71.5 | 62.4 | 61.6 | 59.5 | 58.7 | 58.3 | 58.2 | 58.2 | 58.2 |

The network has learned from all series that a sharp rise like 2025's tends to continue for a year. It then pulls the series back and settles near 58%, a level where its predicted change is close to zero.

**Strengths.** Learns from all 95 series at once, so each series is read against how others moved after similar years; the most accurate model outside the tree models (MAE 15.22).

**Limits here.** 190 rows is very little for a neural network, and the result depends on the random starting weights, so five are averaged. Two years is a short memory, and errors build up as forecasts are fed back in.

## 12 The SARIMA-LSTM hybrid in detail

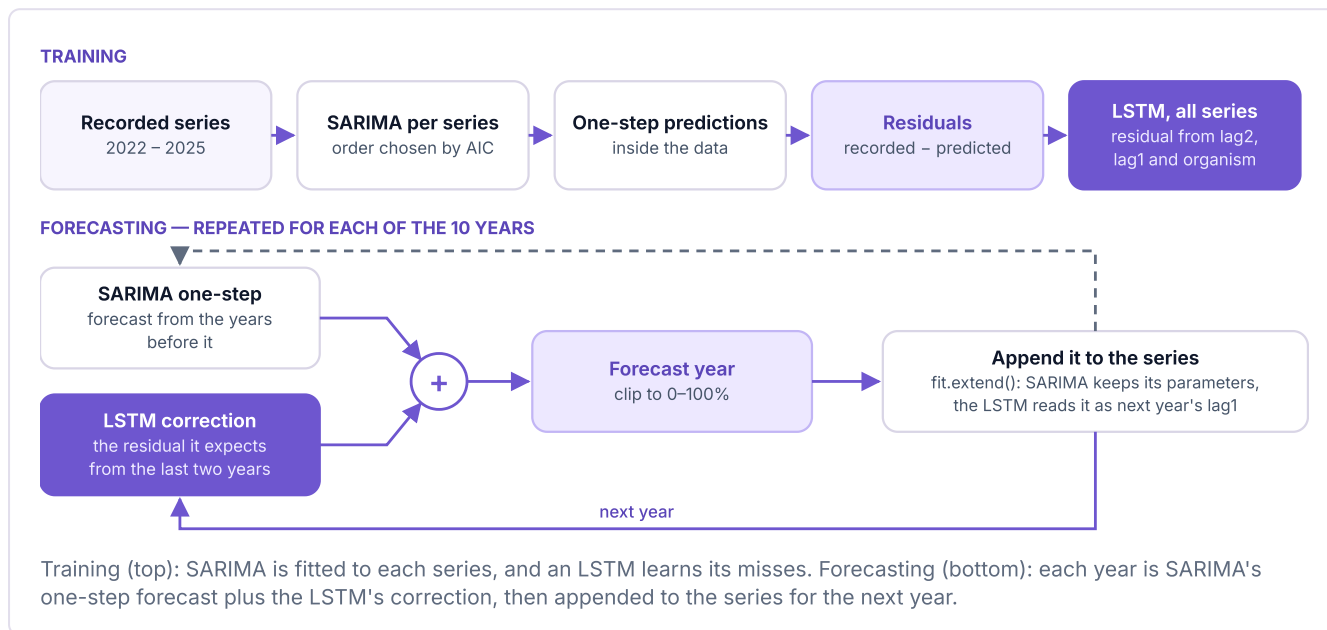
Hybrid

SARIMA per series + one LSTM · Studio only

Backtest MAE 16.97 · RMSE 21.01 · rank 8 of 12

≈ 60 s for 95 series

The hybrid follows Zhang (2003): a series is a linear part, which SARIMA models well, plus whatever SARIMA systematically misses, which a neural network can learn. Each forecast is SARIMA's forecast plus the LSTM's correction.



### Training, step by step

1. **SARIMA for each series**, its form chosen by AIC exactly as in the previous section.
2. **SARIMA's one-step predictions inside the data**: for each recorded year, what SARIMA would have forecast from the years before it.
3. **Residuals** = recorded – predicted, for every year that has two years before it (2024 and 2025): 190 residuals.
4. **An LSTM**, set up exactly like the LSTM model (16 units, 400 passes, 5 seeds averaged), learns to predict the residual from the two previous years and the organism.

### Forecasting, one year at a time

```
studio_models.py — _hybrid()

for j in range(steps):
    linear = np.array([fit.forecast(1)[0] for fit in fits])          # SARIMA, one year ahead
    nxt = np.clip(linear + net.predict(lag_features(last, prev, orgs, levels)), 0, 100)
    path.append(nxt)
    prev, last = last, nxt
    if j < steps - 1:
        fits = [fit.extend(np.array([v])) for fit, v in zip(fits, nxt)] # same parameters, one more year
```

Each forecast year is made the same way the residuals were measured: SARIMA's *one-step* forecast from the years before it, plus the residual the LSTM expects. The year then joins the series for both models' next step. An earlier version added the corrections to SARIMA's multi-step forecast instead; that set every correction against the last recorded year rather than the year before, and two in three series zigzagged.

## Worked example: Meropenem in *E. coli*

SARIMA chose a random walk without drift for this series, so its one-step prediction is always the previous year. Inside the data it predicted 6.82 for 2024 (recorded 11.36) and 11.36 for 2025 (recorded 31.91): residuals of **+4.54** and **+20.55**, two of the 190 the LSTM learned from.

| Year              | 2026  | 2027 | 2028  | 2029 | 2030 | 2031 | 2032 | 2033 | 2034 | 2035 |
|-------------------|-------|------|-------|------|------|------|------|------|------|------|
| SARIMA one-step   | 31.9  | 57.1 | 63.9  | 50.7 | 44.8 | 46.1 | 49.7 | 50.7 | 49.2 | 48.2 |
| + LSTM correction | +25.2 | +6.8 | -13.2 | -5.9 | +1.3 | +3.6 | +1.0 | -1.4 | -1.1 | +0.2 |
| = Forecast        | 57.1  | 63.9 | 50.7  | 44.8 | 46.1 | 49.7 | 50.7 | 49.2 | 48.2 | 48.4 |

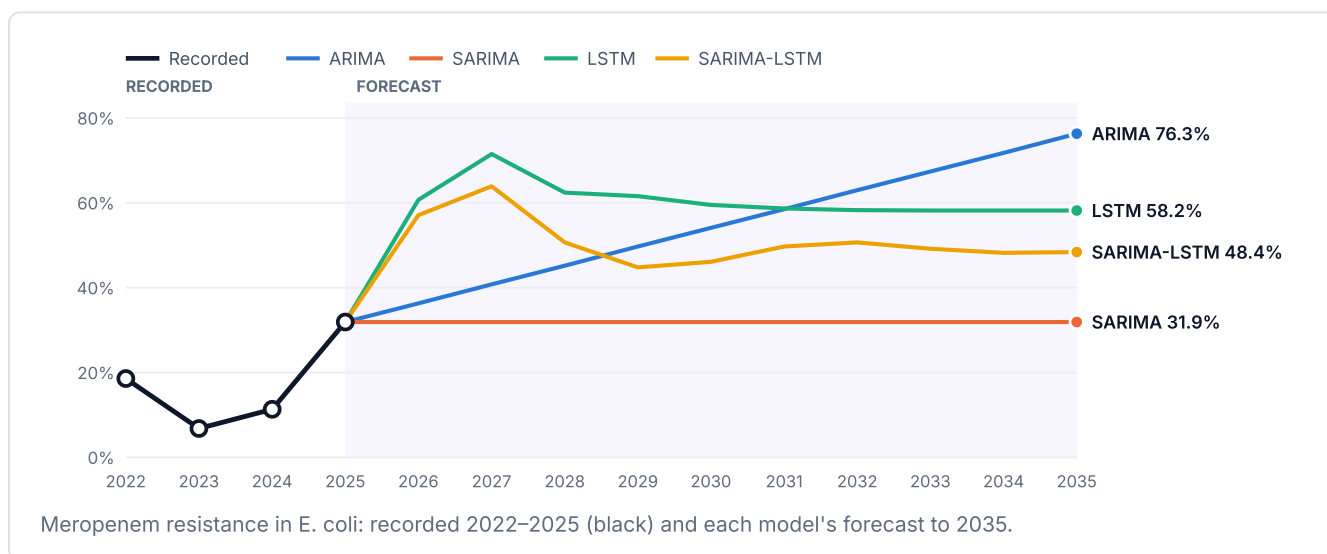
The LSTM adds a large correction in the first year, because SARIMA's flat line misses rises like 2025's. The correction shrinks, and turns negative after the overshoot in 2027, until the series settles near 48%.

**Strengths.** Keeps SARIMA's per-series fit while borrowing what the LSTM learned across all series. Its MAE (16.97) is a little better than SARIMA alone (17.04).

**Limits here.** Its RMSE (21.01) is a little worse than SARIMA's (20.79): its large misses are larger. It is among the slowest to train, with the limits of both its parts.

## 13 The four models on one medicine

The same four recorded values, four quite different futures.



| Model         | How it reads 18.6 → 6.8 → 11.4 → 31.9                  | 2026 | 2027 | 2030 | 2035 | MAE   |
|---------------|--------------------------------------------------------|------|------|------|------|-------|
| ● ARIMA       | An average rise of 4.4 pts a year, extended            | 36.3 | 40.8 | 54.1 | 76.3 | 17.26 |
| ● SARIMA      | Too uneven to justify a trend: stays at the last value | 31.9 | 31.9 | 31.9 | 31.9 | 17.04 |
| ● LSTM        | A sharp rise usually continues, then levels off        | 60.7 | 71.5 | 59.5 | 58.2 | 15.22 |
| ● SARIMA-LSTM | The flat line, corrected for what it misses            | 57.1 | 63.9 | 46.1 | 48.4 | 16.97 |

**Reading a disagreement.** No single model is “right” here. In the backtest none of these four beat repeating the previous year, so the spread between them is itself the useful signal: the further apart the models are, the less certain the future. This is why the Studio lets you tick several models and export a comparison report.

## 14 The other eight models in brief

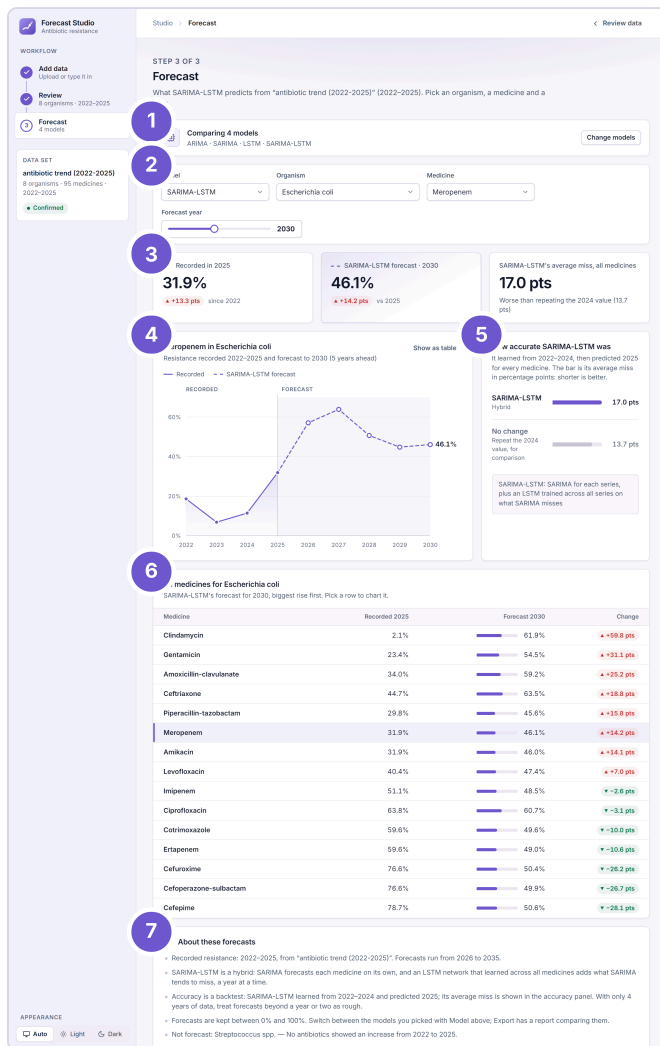
The remaining models are shared with the original Forecast app. All are trained and tested exactly as described in sections 7 and 8.

| Model                 | Family           | How the Studio sets it up                                                                                                                                                                                                                                   | MAE   |
|-----------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Exponential Smoothing | Statistical      | Holt's damped linear trend, fitted to each series. The weights are fixed (level 0.8, trend 0.2, damping 0.9) because 3–4 points are too few to estimate them; it starts from the first value and first change.                                              | 16.99 |
| Theta                 | Statistical      | The Theta method (statsmodels <code>ThetaModel</code> , no seasonal adjustment): a drift line blended with simple exponential smoothing.                                                                                                                    | 15.82 |
| Prophet               | Statistical      | Meta's trend model with automatic changepoints, seasonality off, on yearly dates. It runs a compiled Stan program for each fit (190 per training); the app copies Prophet's own <code>tbb.dll</code> beside it so an older copy in System32 can't break it. | 17.50 |
| Random Forest         | Machine learning | 500 decision trees, at least 3 rows per leaf, trained across all series on the lag features; predicts next year's change.                                                                                                                                   | 12.74 |
| XGBoost               | Machine learning | 300 gradient-boosted trees of depth 2, learning rate 0.03, each tree on 80% of the rows and columns.                                                                                                                                                        | 12.99 |
| LightGBM              | Machine learning | 300 leaf-wise boosted trees of 4 leaves, learning rate 0.03, at least 5 rows per leaf, 80% subsampling.                                                                                                                                                     | 13.02 |
| CatBoost              | Machine learning | 500 iterations of ordered boosting, depth 3, learning rate 0.03.                                                                                                                                                                                            | 13.18 |
| GRU                   | Deep learning    | The LSTM model's set-up with a gated recurrent unit, a simpler cell with two gates (1,265 weights per network); five seeds averaged.                                                                                                                        | 15.36 |

The tree models use a fixed random seed (42) and the networks seeds 0–4, so every training gives the same forecasts.

## 15 Reading and exporting the forecasts

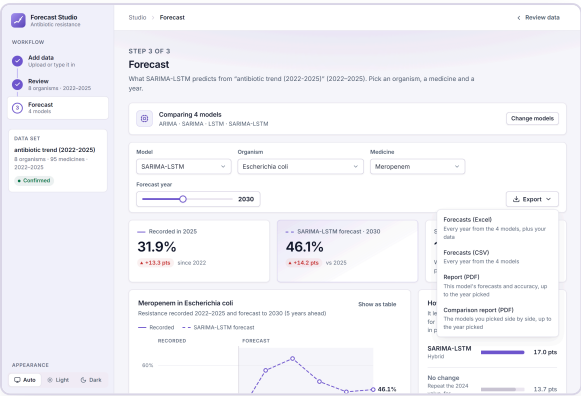
Once a model is trained, Step 3 becomes a dashboard for it. When several models were picked, the **Model** menu switches between them, marking the most accurate.



- Models.** Which models are shown, and *Change models* to reopen the picker.
- Controls.** Model, organism, medicine, and a slider for the forecast year (2026–2035).
- Tiles.** The last recorded value and its change since the first year; the forecast for the year picked and its change since the last recorded year; the model's average miss against repeating the year before.
- Chart.** Recorded values (solid) and the forecast (dashed) in a shaded forecast zone. Hover, or the arrow keys, read each year; *Show as table* lists them.
- Accuracy.** The model's backtest miss beside the no-change baseline, and how the model works.
- All medicines** for the organism, biggest predicted rise first. Clicking a row charts it.
- Notes.** Where the data came from, how this kind of model works, how far to trust it, and any organism not forecast.

The page loads `/api/meta` (recorded values, models, accuracy) and `/api/forecast?year=2035` once. Switching model, organism, medicine or year is then worked out in the page from those two answers, so it is instant. The choices are saved in the session, and the dashboard reopens as it was left.

Exports



The Export menu. Comparison report appears when two or more models are picked.

| Export                  | Contents                                                                                                                                                                           |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Forecasts (Excel)       | Three sheets: <i>Forecasts</i> (a row per organism, medicine and model; a column per year up to the one picked), <i>Recorded</i> , and <i>Model accuracy</i> (MAE, RMSE, baseline) |
| Forecasts (CSV)         | The Forecasts table, UTF-8 with a byte-order mark so Excel reads names correctly                                                                                                   |
| Report (PDF)            | The model on screen: accuracy against the baseline, how it works, the forecasts at a glance, a page per organism, notes                                                            |
| Comparison report (PDF) | The picked models side by side: accuracy ranking, where each heads, a column per model on each organism page                                                                       |

`DesktopApi.choose_export_path` opens the Windows Save dialog with a suggested name (for example *Resistance forecast report 2026-2030 SARIMA-LSTM.pdf*); `export()` then writes only to a path chosen in that dialog, one export at a time. The PDFs are laid out as HTML and printed by headless Edge or Chrome. The page's **Open** link opens only files the app itself wrote. Exports need the desktop window: in a plain browser, the menu is hidden.

## PART C

# How the app is made

The technology, the desktop window and local server, the page, where data is kept, and how the Windows installer is built and tested.

## 16 Technology stack

The Studio is Python from the window down, with one HTML page for the interface. Versions are pinned in `requirements.txt` and `requirements-build.txt`, so a rebuild gives the same forecasts.

| Layer              | Technology                                                                | Used for                                                                                             |
|--------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Language           | Python 3.11 (the project's <code>.venv</code> )                           | Everything outside the page                                                                          |
| Window             | pywebview 6.2.1 on Microsoft Edge WebView2                                | A native window showing the page; Open and Save dialogs; the JavaScript ↔ Python bridge              |
| Interface          | HTML, CSS and plain JavaScript in one file; no framework or chart library | The three steps, the editable grid, the SVG chart, autosave; Inter font embedded so it works offline |
| Server             | FastAPI 0.136.1 and uvicorn 0.46.0                                        | The JSON API on 127.0.0.1                                                                            |
| Data               | pandas 2.2.3, NumPy 2.4.6, openpyxl 3.1.5                                 | Reading and writing workbooks; the series × years arrays                                             |
| Statistical models | statsmodels 0.15.0; Prophet 1.4.0 (Stan)                                  | ARIMA, SARIMA, Exponential Smoothing, Theta; Prophet                                                 |
| Machine learning   | scikit-learn 1.8.0, XGBoost 3.2.0, LightGBM 4.7.0, CatBoost 1.2.10        | Random Forest and the boosted trees; the accuracy scores                                             |
| Deep learning      | PyTorch 2.14.1 (CPU)                                                      | LSTM, GRU, and the hybrid's LSTM                                                                     |
| Reports            | HTML printed by headless Edge or Chrome                                   | The two PDF reports                                                                                  |
| Packaging          | PyInstaller 6.22.3; Inno Setup 6                                          | The app folder; the installer                                                                        |

## One code base, two apps

The Studio shares its code with the original *Antibiotic Resistance Forecast* app. A **profile** chosen at start-up ( `appdata.PROFILES` ) decides which app runs:

|                  | Forecast app ( <code>forecast</code> )                        | Studio ( <code>entry</code> )                                                                                       |
|------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Starts with      | The bundled sample workbook, models pre-trained at build time | Nothing: the user adds data                                                                                         |
| Page             | <code>static/index.html</code> (blue)                         | <code>static/studio.html</code> (lavender)                                                                          |
| Models           | All 10, trained together                                      | Any of 12 (adds SARIMA and SARIMA-LSTM), trained only when picked                                                   |
| Data folder      | <code>%LOCALAPPDATA%\Antibiotic Resistance Forecast</code>    | <code>%LOCALAPPDATA%\Antibiotic Resistance Forecast Studio</code>                                                   |
| Studio endpoints | Answer 404                                                    | <code>/api/session</code> , <code>/api/confirm</code> , <code>/api/models</code> , <code>/api/generate</code> , ... |

## 17 The desktop window and the local server

### The desktop shell — `desktop.py`

The shell turns a web app into a desktop app. Besides the start-up steps in section 2, it gives the page a bridge object, `DesktopApi` , which the page calls as `window.pywebview.api.<method>()` :

| Method                                                 | What it does                                                                    |
|--------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>read_workbook_for_review()</code>                | Windows Open dialog for an .xlsx, then reads it for Step 2                      |
| <code>save_data_excel(dataset)</code>                  | Saves the reviewed data as a workbook in the source layout ("Save as Excel...") |
| <code>choose_export_path(kind, year, models)</code>    | Windows Save dialog with a suggested file name                                  |
| <code>export(kind, year, path, models)</code>          | Writes the Excel, CSV or PDF export to the chosen path                          |
| <code>open_path(path)</code> · <code>open_log()</code> | Opens an exported file, or the log, in Windows                                  |

The bridge is deliberately narrow: exports go only to paths the user picked in a dialog, only files the app wrote can be opened, only one export runs at a time, and methods whose names start with `_` are hidden from the page.

## The server — `app.py`

| Endpoint                             | Purpose                                                                                                           |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| GET <code>/</code>                   | The page ( <code>studio.html</code> ), never cached                                                               |
| GET · PUT <code>/api/session</code>  | Where the user left off: step, data set, theme, view. PUT saves and returns the data set's problems               |
| POST <code>/api/upload?name=</code>  | An .xlsx file's values for review (used in a browser)                                                             |
| POST <code>/api/workbook</code>      | A data set as an .xlsx file (Save as Excel in a browser)                                                          |
| POST <code>/api/confirm</code>       | Check and activate a data set (section 5); trains nothing                                                         |
| GET <code>/api/models</code>         | The 12 models: name, family, description, typical seconds                                                         |
| POST <code>/api/generate</code>      | <code>{"models": [...]}</code> : queue training for those not yet Ready                                           |
| GET <code>/api/status</code>         | Idle, training (model, step, progress, elapsed), ready or failed; which models are Ready                          |
| GET <code>/api/meta</code>           | Recorded values, models with MAE and RMSE, best model, year range, organisms not forecast                         |
| GET <code>/api/forecast?year=</code> | Each model's forecasts up to that year; filter with <code>&amp;organism=</code> and <code>&amp;antibiotic=</code> |
| GET <code>/docs</code>               | Interactive API documentation, generated by FastAPI                                                               |

The Studio-only endpoints carry a `studio_only` dependency, so the same server running as the Forecast app answers them with 404. Heavy imports (PyTorch, Prophet, the boosting libraries) happen only when training starts, so the server is listening within moments of launch.

## The Trainer's states

`idle` → `training` → `ready`, or `failed` with the error and the model that failed. While a model trains, the models already trained keep answering `/api/meta` and `/api/forecast`. A new engine replaces the old one only when it is complete, and a failure keeps the old one.

## 18 The page — studio.html

The whole interface is one HTML file of about 2,500 lines: styles, icons, the embedded Inter font and the script. It needs no build step and no internet connection.

### State

- **S**: the step, the draft data set, the theme, whether the draft came from a file or changed since the forecasts, the forecast view, the problems, the latest status.
- **F**: the dashboard: meta, forecasts, and the organism, medicine, model and year shown.
- **P**: the picker: the 12 models, the ticked ones, whether it is open, the last error.

### Start-up and steps

`boot()` fetches status, session and models in parallel, applies the theme, builds the picker and calls `goTo(step)`. `goTo` won't open Review without data or Forecast without confirmed data, so a stale session can never show an empty step.

### Talking to the server

Edits are saved 0.3 s after the last keystroke ( `saveSoon` ); a sequence number discards a stale reply. Training is followed by polling `/api/status` every 0.6 s; if the server stops answering, the page says so and retries every 2 s.

### Look and feel

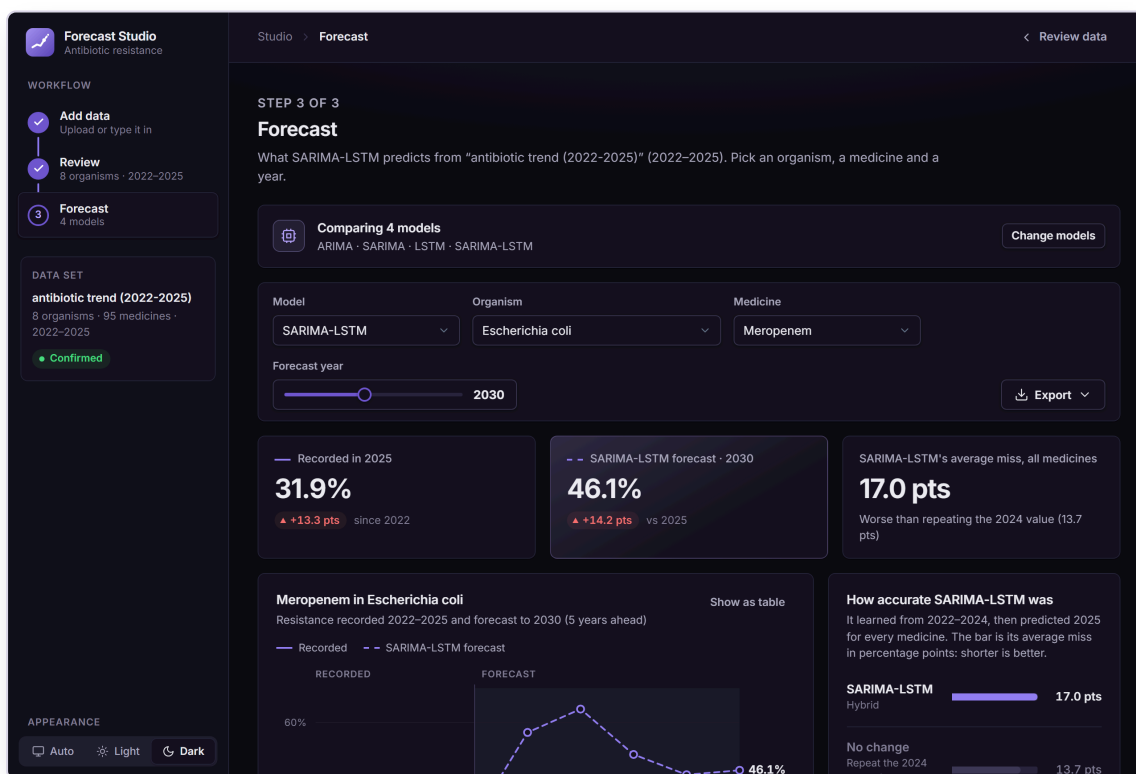
A sidebar app shell (brand, the three steps, the data set card, appearance), “clinical slate” neutrals with a lavender accent ( `#6e56cf` ), and Inter. It is deliberately distinct from the blue Forecast app. **Auto** follows the Windows light/dark setting; **Light** and **Dark** override it and are remembered.

### Accessible

Steps, tabs, menus and dialogs use ARIA roles; the chart can be read with the arrow keys or shown as a table; problems are announced as well as coloured.

### In a browser too

`python app.py --profile entry` serves the same page to a browser. Without `window.pywebview` it uploads through `/api/upload` and saves through `/api/workbook`, and hides the exports.



The same dashboard in Dark mode. The chart is redrawn with the dark theme's own colours.

## 19 Where the Studio keeps its data

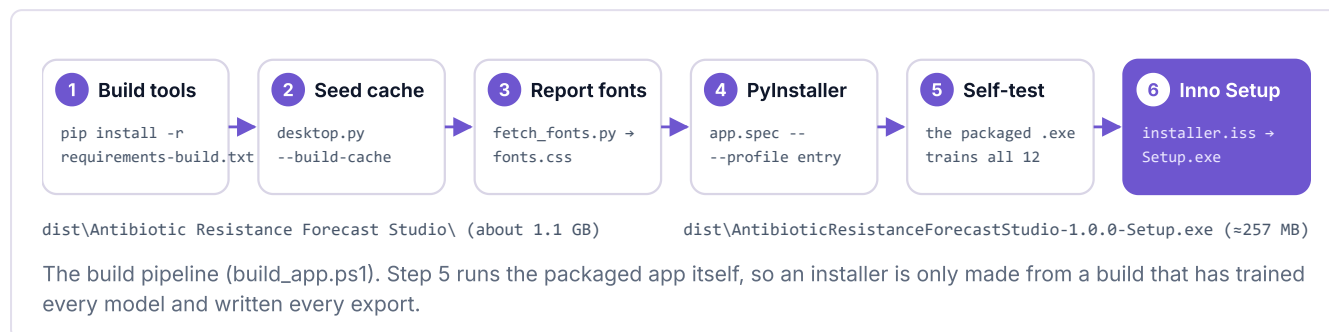
Everything the Studio writes goes to one folder per Windows user. The program folder is never written to, so it can live in Program Files.

```
%LOCALAPPDATA%\Antibiotic Resistance Forecast Studio\  
├ session.json          the step, the data set being edited, theme and forecast view  
├ workbook\  
│   ├── state.json      the active data set: its name, where it came from, its key  
│   └── <key>.xlsx      each confirmed data set, as a workbook  
├ cache\  
│   ├── <key>\          trained models for one data set (the newest 5 are kept)  
│   │   ├── meta.json   models, MAE and RMSE, years, recorded values  
│   │   ├── paths.npy   forecasts [series, model, year]  
│   │   └── recorded.npy the recorded values, unrounded  
├ logs\app.log          everything the app prints (rolled over at 5 MB)  
├ tmp\  
└ webview\  
    the window's browser profile
```

- **Safe saves.** Sessions and caches are written to a temporary name first and then renamed, so a crash never leaves half a file.
- **Pruning.** After each save, all but the 5 most recently used caches (and their workbooks) are deleted, so the folder doesn't grow without limit.
- **Damage-tolerant.** An unreadable session starts fresh, and a damaged cache is simply retrained.
- **Uninstall** removes the folder: it holds only copies, caches and logs.
- `ARF_DATA_DIR` points the app at another folder instead. The build's self-test uses it, and so did the screenshots in this guide, so the real Studio data was untouched.

## 20 Building and packaging the installer

One command builds the Studio's Windows installer: `.\build_app.ps1 -App Studio`. It runs six steps and stops at the first failure.



1. **Build tools** are installed into `.venv` from `requirements-build.txt` (pywebview, PyInstaller and the hooks, on top of the app's requirements).
2. **Seed cache**. The 10 shared models are trained on the sample workbook. The Forecast app ships these; the Studio, which starts empty, uses them only to check its self-test.
3. **Report fonts** are saved to `assets\fonts\fonts.css`, so the PDF reports work offline.
4. **PyInstaller** packages the app with `packaging\app.spec -- --profile entry`: the script `entry_desktop.py`, the page `studio.html`, and the lavender icon `entry.ico`.
5. **Self-test** of the packaged `.exe`, with a throwaway data folder: reads the sample as an upload, confirms it, trains all 12 models, checks the model list, checks that models trained alone match those trained together, compares the 10 shared models with the seed cache (difference must be 0.00), and writes every kind of export. The exit code decides whether the build continues.
6. **Inno Setup** compiles `packaging\installer.iss` into `dist\AntibioticResistanceForecastStudio-1.0.0-Setup.exe`.

### PyInstaller details

- A folder build (an `.exe` plus `_internal\`) with no console window.
- Bundles Prophet's compiled Stan model, with its `tbb.dll` copied beside it at build time (Program Files is read-only once installed), and XGBoost's libraries.
- Leaves out what the app never imports (matplotlib, plotly, Pillow, tkinter, IPython, test folders) and PyTorch's C++ headers and link libraries. It **keeps sympy**, which PyTorch imports during the networks' first training step.
- No UPX compression: it corrupts PyTorch's DLLs and makes antivirus tools suspicious.

### The installer

- Installs for the current user by default, with no administrator rights; the first page offers all users.
- Windows 10 or later, 64-bit. Warns, and offers Microsoft's download page, if WebView2 is missing.
- Start-menu shortcut, optional desktop icon, LZMA2 compression.
- A fixed AppId, `{E04C8B64-4ED7-4B2B-98F5-43D105BEF67D}`, so new versions upgrade in place and the Studio installs alongside the Forecast app.
- Size: about **1.1 GB** installed, mostly PyTorch and the model libraries; the installer is about **257 MB**.

## 21 Running it from source

Use the project's Python 3.11 virtual environment. The default `python` on this PC (conda 3.13) doesn't have the forecasting libraries.

```
py -3.11 -m venv .venv
.venv\Scripts\pip install -r requirements-build.txt

.venv\Scripts\python entry_desktop.py           # the Studio in its own window
.venv\Scripts\python app.py --profile entry      # the Studio in a browser (http://127.0.0.1:8000)
.venv\Scripts\python entry_desktop.py --self-test --workbook "antibiotic trend (2022-2025).xlsx" --seed-cache
build\seed_cache

.\build_app.ps1 -App Studio                     # the installer, in dist\
.\build_app.ps1 -App Studio -SkipInstaller      # stop after the self-test
```

After changing a model or its settings, raise `CACHE_VERSION` in `forecast_engine.py` so caches made with the old settings are retrained, and rebuild.

## 22 Glossary

|                    |                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| <b>AIC</b>         | Akaike information criterion: a model's fit penalised by its number of parameters. SARIMA picks the lowest. |
| <b>Backtest</b>    | Training without the last recorded year and predicting it, to measure accuracy.                             |
| <b>Baseline</b>    | Predicting no change (repeat the previous year). A model should beat it.                                    |
| <b>Cache key</b>   | A fingerprint of the confirmed values (plus a version number) naming the folder of models trained on them.  |
| <b>Drift</b>       | A constant average yearly change, added every year by ARIMA (and by SARIMA when AIC chooses it).            |
| <b>Epoch</b>       | One pass of a network's training over all its training rows. The LSTM uses 400.                             |
| <b>Lag</b>         | A previous year's value used as an input: lag1 is the year before, lag2 the year before that.               |
| <b>MAE · RMSE</b>  | Average absolute miss, and root-mean-square miss (which weighs large misses more), in percentage points.    |
| <b>One-hot</b>     | Encoding a category as flags: one column per organism, 1 for this series' organism and 0 for the rest.      |
| <b>Random walk</b> | A series whose next value is its last value plus noise: forecast flat.                                      |
| <b>Residual</b>    | Recorded minus a model's prediction: what the model missed. The hybrid's LSTM learns these.                 |
| <b>Seed</b>        | The number that fixes a model's random starting point, so training repeats exactly.                         |
| <b>Series</b>      | One organism–antibiotic pair's yearly resistance percentages. The sample has 95.                            |
| <b>WebView2</b>    | Microsoft's Edge-based component that shows web content inside Windows apps; the Studio's window.           |